

Lab Assignment 7

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC



Submitted By:

Ryan Kapoor

Roll No. 1024150188

Batch: 2023

1.)

Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

Input:

```
#include <iostream>
using namespace std;

class Polygon {
public:
    int width, height;

    void set_value(int w, int h) {
        width = w;
        height = h;
    }

    virtual int calculate_area() = 0;
};

class Rectangle : public Polygon {
public:
    int calculate_area() {
        return width * height;
    }
};

class Triangle : public Polygon {
public:
    int calculate_area() {
        return (width * height) / 2;
    }
};

int main() {
```

```

Polygon *p;

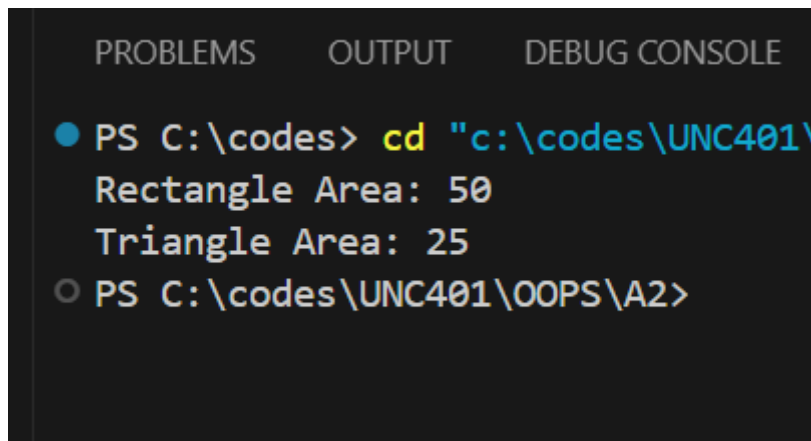
Rectangle r;
Triangle t;

p = &r;
p->set_value(10, 5);
cout << "Rectangle Area: " << p->calculate_area() << endl;

p = &t;
p->set_value(10, 5);
cout << "Triangle Area: " << p->calculate_area() << endl;

return 0;
}

```



```

PROBLEMS OUTPUT DEBUG CONSOLE
● PS C:\codes> cd "c:\codes\UNC401\
Rectangle Area: 50
Triangle Area: 25
○ PS C:\codes\UNC401\OOPS\A2>

```

2.)

Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() {
        cout << "Area of Shape" << endl;
    }

    virtual void display() {
        cout << "This is a shape" << endl;
    }
};

class Circle : public Shape {
public:
    float r;

    Circle(float radius) {
        r = radius;
    }

    void area() {
        cout << "Area of Circle: " << 3.14 * r * r << endl;
    }

    void display() {
        cout << "Shape: Circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    float l, b;

    Rectangle(float length, float breadth) {
        l = length;
        b = breadth;
    }

    void area() {
        cout << "Area of Rectangle: " << l * b << endl;
    }

    void display() {
        cout << "Shape: Rectangle" << endl;
    }
};

class Triangle : public Shape {
public:
    float b, h;

    Triangle(float base, float height) {
        b = base;
        h = height;
    }

    void area() {
```

```
        cout << "Area of Triangle: " << 0.5 * b * h << endl;
    }

    void display() {
        cout << "Shape: Triangle" << endl;
    }
};

int main() {

    Shape *s;

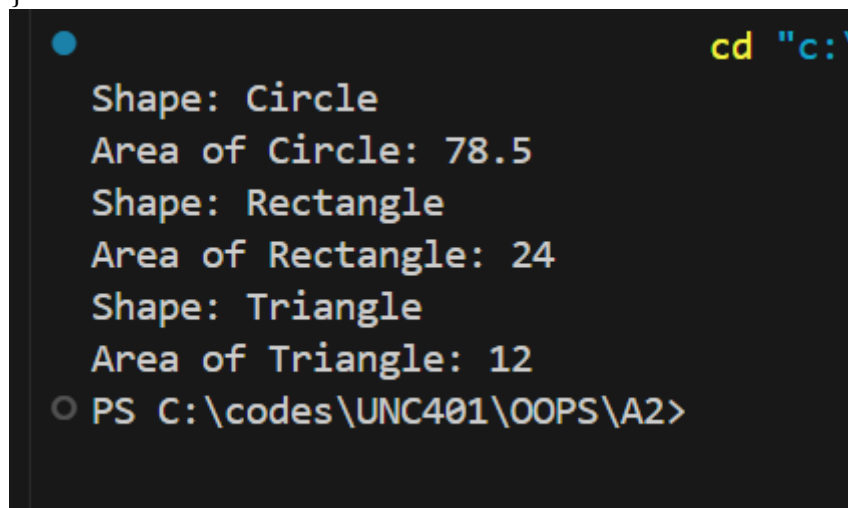
    Circle c(5);
    Rectangle r(4, 6);
    Triangle t(3, 8);

    s = &c;
    s->display();
    s->area();

    s = &r;
    s->display();
    s->area();

    s = &t;
    s->display();
    s->area();

    return 0;
}
```

A screenshot of a Windows command prompt window with a dark background. The prompt is 'PS C:\codes\UNC401\OOPS\A2>'. The output of the program is displayed in a monospaced font: 'Shape: Circle', 'Area of Circle: 78.5', 'Shape: Rectangle', 'Area of Rectangle: 24', 'Shape: Triangle', and 'Area of Triangle: 12'. The prompt 'PS C:\codes\UNC401\OOPS\A2>' is shown again at the bottom. The window title bar is partially visible at the top, showing 'cd "c:\'.

3.)

Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

```
#include <iostream>
#include <cmath>
using namespace std;

class Triangle {
public:

    float area(float base, float height) {
        return 0.5 * base * height;
    }

    float area(float side) {
        return (sqrt(3) / 4) * side * side;
    }

    float area(float a, float b, float c) {
        float s = (a + b + c) / 2;
        return sqrt(s * (s - a) * (s - b) * (s - c));
    }
};

int main() {

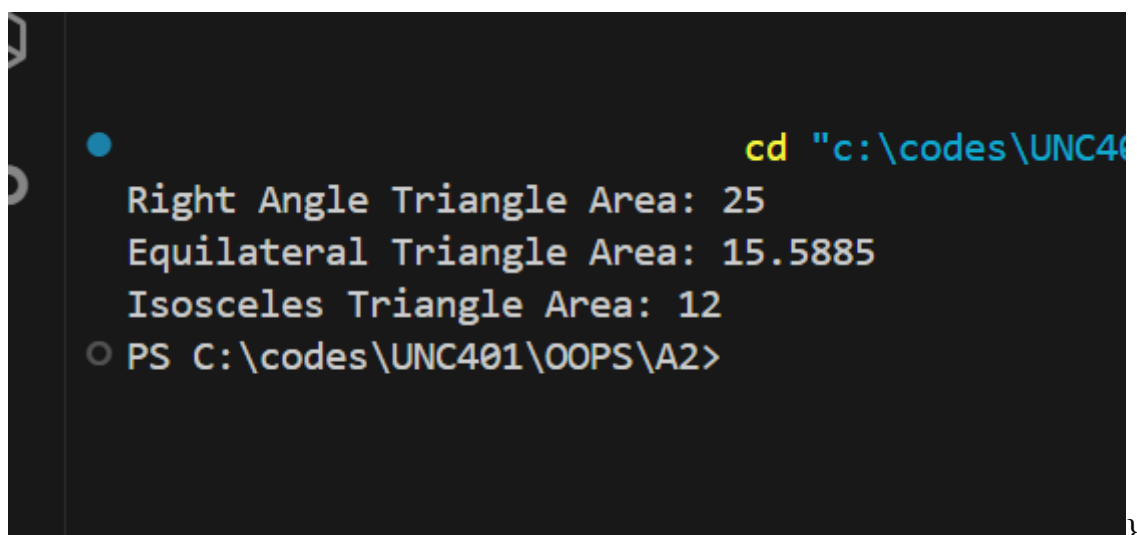
    Triangle t;

    cout << "Right Angle Triangle Area: " << t.area(10, 5) << endl;

    cout << "Equilateral Triangle Area: " << t.area(6) << endl;

    cout << "Isosceles Triangle Area: " << t.area(5, 5, 6) << endl;

    return 0;
```

A screenshot of a terminal window with a dark background. The prompt is 'PS C:\codes\UNC401\OOPS\A2>'. The output shows three lines: 'Right Angle Triangle Area: 25', 'Equilateral Triangle Area: 15.5885', and 'Isosceles Triangle Area: 12'. The prompt is repeated at the bottom. There are some small icons on the left side of the terminal window.

```
cd "c:\codes\UNC401\OOPS\A2"
Right Angle Triangle Area: 25
Equilateral Triangle Area: 15.5885
Isosceles Triangle Area: 12
PS C:\codes\UNC401\OOPS\A2>
```

4.)

Write a program with Student as abstract class and create derived classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {  
public:  
    virtual void display() = 0;  
};
```

```
class Engineering : public Student {  
public:  
    void display() {  
        cout << "Student of Engineering" << endl;  
    }  
};
```

```
class Medicine : public Student {  
public:  
    void display() {  
        cout << "Student of Medicine" << endl;  
    }  
};
```

```
class Science : public Student {  
public:  
    void display() {  
        cout << "Student of Science" << endl;  
    }  
};
```

```
int main() {  
  
    Student* arr[3];  
  
    Engineering e;  
    Medicine m;  
    Science s;  
  
    arr[0] = &e;  
    arr[1] = &m;  
    arr[2] = &s;  
  
    for(int i = 0; i < 3; i++) {  
        arr[i]->display();  
    }  
  
    return 0;  
}
```

```
cd "c:\codes"
Student of Engineering
Student of Medicine
Student of Science
PS C:\codes\UNC401\OOPS\A2>
```

5.)

Create a class Time with three private variables h, m, s; Create a function to overload '+' operator to add two time variables.

```
int main(){
    Time t1(5,15,34), t2(9,53,58), t3;
    t3 = t1 + t2;
    t3.show();
}
```

```
#include <iostream>
using namespace std;
```

```
class Time {
private:
    int h, m, s;
```

```
public:
    Time() {
        h = m = s = 0;
    }
```

```
    Time(int h, int m, int s) {
        this->h = h;
        this->m = m;
        this->s = s;
    }
```

```
    Time operator+(Time t) {
        Time temp;

        temp.s = s + t.s;
        temp.m = m + t.m + temp.s / 60;
        temp.s = temp.s % 60;

        temp.h = h + t.h + temp.m / 60;
        temp.m = temp.m % 60;

        return temp;
    }
```

```
    }

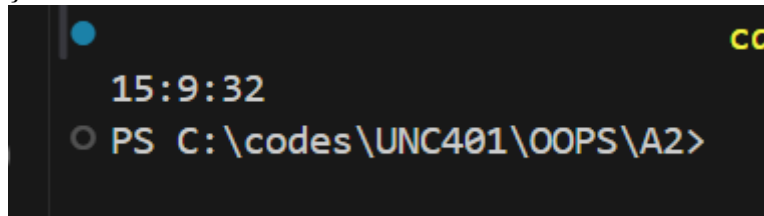
    void show() {
        cout << h << ":" << m << ":" << s << endl;
    }
};

int main(){

    Time t1(5,15,34), t2(9,53,58), t3;

    t3 = t1 + t2;
    t3.show();

    return 0;
}
```



6.)

Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
class STRING {
```

```
    char str[100];
```

```
public:
```

```

    STRING(const char* s = "") { strcpy(str, s); }

    bool operator==(STRING s) {
        return strcmp(str, s.str) == 0;
    }

    STRING operator+(STRING s) {
        STRING temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }

    void display() { cout << str << endl; }
};

int main() {
    STRING s1("Hello "), s2("World");

    STRING s3 = s1 + s2;

    s3.display();

    if (s1 == s2) cout << "Equal";
    else cout << "Not Equal";
}

```

```
Output
Hello World
Not Equal

=== Code Execution Successful ===
```

7.)

Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

```
#include <iostream>
```

```
using namespace std;
```

```
class Matrix {
```

```
    int a[3][3];
```

```
public:
```

```
    void input(){
```

```

        for(int i=0;i<3;i++)
            for(int j=0;j<3;j++)
                cin>>a[i][j];
    }

    void display(){
        for(int i=0;i<3;i++){
            for(int j=0;j<3;j++){
                cout<<a[i][j]<<" ";
            }
            cout<<endl;
        }
    }

    friend Matrix operator*(Matrix m1, Matrix m2);
};

Matrix operator*(Matrix m1, Matrix m2){
    Matrix temp;
    for(int i=0;i<3;i++){
        for(int j=0;j<3;j++){
            temp.a[i][j]=0;
            for(int k=0;k<3;k++){
                temp.a[i][j]+=m1.a[i][k]*m2.a[k][j];
            }
        }
    }
    return temp;
}

int main(){
    Matrix m1,m2,m3;

    cout<<"Enter Matrix 1:\n";

    m1.input();

    cout<<"Enter Matrix 2:\n";

    m2.input();

```

```
m3 = m1 * m2;  
cout<<"Result:\n";  
m3.display();  
}
```

```
Output Clear
Enter Matrix 1:
1 0 0
0 0 1
2 3 5
Enter Matrix 2:
7 5 2
1 9 3
2 7 1
Result:
7 5 2
2 7 1
27 72 18

=== Code Execution Successful ===
```

8.)

Overload '[' to check array index out of bounds problem at run time.

```
#include <iostream>

using namespace std;

class Array {
    int a[5];
public:
    Array() {
        for(int i=0;i<5;i++)
            a[i] = i+1;
    }
    int operator[](int i) {
        if(i < 0 || i >= 5) {
            cout << "Index out of bounds" << endl;
            return -1;
        }
        return a[i];
    }
}
```

```
};  
  
int main() {  
    Array obj;  
    cout << obj[2] << endl;  
    cout << obj[10] << endl;  
}
```

9.)

Overload ‘()’ to input arbitrary number of input arguments for an object.

```
#include <iostream>  
  
using namespace std;  
  
class Input {  
public:
```

Output Clear

```
3  
Index out of bounds  
-1  
  
=== Code Execution Successful ===
```

```
void operator()(int n) {  
    int x;  
    cout << "Enter " << n << " numbers:" << endl;  
    for(int i=0;i<n;i++) {  
        cin >> x;  
    }  
}  
};  
int main() {  
    Input obj;  
    obj(3);  
}
```

10.)

Write a program in C++ to overload input operator (>>) and output operator (<<).

```
Output Clear  
Enter 3 numbers:  
1 2 3  
  
=== Code Execution Successful ===
```

```
#include <iostream>

using namespace std;

class Test {
    int x;
public:
    friend istream& operator>>(istream &in, Test &t) {
        in >> t.x;
        return in;
    }
    friend ostream& operator<<(ostream &out, Test &t) {
        out << t.x;
        return out;
    }
};

int main() {
    Test t;
    cout << "Enter value: ";
    cin >> t;
    cout << "Output: " << t;
}
```

Output

Clear

Enter value: 2
Output: 2

=== Code Execution Successful ===

11.)

Write a program to convert basic data type (float) to user defined data type

(object). *class Test {*
private:
 //...
public:
 Test(data_type) {
 // conversion code
 }
};

#include <iostream>

using namespace std;

class Test {
 float x;

public:
 Test(float f) {
 x = f;
 }

```
    }  
  
    void show() {  
        cout << "Value: " << x;  
    }  
};  
  
int main() {  
    float f = 5.5;  
    Test t = f;  
    t.show();  
}
```

12.)

Write a program to convert UDT to basic data

```
type (float) class Test {  
public:  
    operator data_type() {  
        // conversion code
```

Output Clear

Value: 5.5

=== Code Execution Successful ===



```
    }  
};  
  
#include <iostream>  
  
using namespace std;  
  
class Test {  
    float x;  
  
public:  
    Test(float a = 0) {  
        x = a;  
    }  
  
    operator float() {  
        return x;  
    }  
};  
  
int main() {  
    Test t(7.5);  
  
    float f;  
  
    f = t;  
  
    cout << "Value: " << f;  
}
```

Output

Clear

Value: 7.5

=== Code Execution Successful ===

13.)

How will you convert one UDT to another UDT. For example conversion of polar to cartesian system.

```
Polar p(10,5);
```

```
Cartesian c = p;
```

```
c.show();
```

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
class Polar {
```

```
    float r, theta;
```

```
public:
```

```
    Polar(float r, float theta) {
```

```
        this->r = r;
```

```
        this->theta = theta;
```

```
    }
```

```
    float getR() { return r; }
```

```
    float getTheta() { return theta; }
```

```
};
```

```
class Cartesian {
```

```
    float x, y;
```

```
public:
```

```
    Cartesian(Polar p) {
```

```
        x = p.getR() * cos(p.getTheta());
```

```
        y = p.getR() * sin(p.getTheta());
```

```
    }  
  
    void show() {  
        cout << "x = " << x << " y = " << y;  
    }  
};  
  
int main() {  
    Polar p(10,5);  
    Cartesian c = p;  
    c.show();  
}
```

Output

[Clear](#)

```
x = 2.83662 y = -9.58924
```

```
=== Code Execution Successful ===
```

